K.R. MANGALAM UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY

PRODUCT FEEDBACK INTELLIGENCE PLATFORM

A Kotlin-primary product feedback and issue intelligence system

SHORT PROJECT DOCUMENTATION

Project overview, objectives, key features, technologies and implementation

Course	ENSP465 - New Age Programming Languages Lab
Student	Anshu Yadav
Roll Number	2401720008
Programme	B.Sc. Computer Science
Semester / Batch	Semester V | Batch 2024–2027
Section	A
Academic Year	2026–2027
Faculty Guide	Dr. Hemraj Kumawat
Submission Date	10 Nov 2026

School of Engineering and Technology
K.R. Mangalam University, Gurugram, Haryana

Abstract and Project Overview

Product teams receive feedback through application reviews, support requests, surveys, feature-request forms and other channels. This information is valuable but often arrives as unstructured, repeated or incomplete text. Manual review makes it difficult to recognize recurring problems, monitor changes and decide which issues deserve investigation first.

The **Product Feedback Intelligence Platform** is planned as a Kotlin-primary application that adapts the workflow foundation of the open-source **Freefair Issue Tracker** into a product-feedback workflow. The existing tracker provides boards, tasks, status columns, backlog categories, tags, search, archive and drag-and-drop organization. These capabilities will form the operational layer for organizing product issues.

Feedback ingestion, NLP/ML analysis, trend monitoring, explainable priority scoring and product-intelligence reporting are proposed extensions. They are **not existing Freefair features**. The system will initially support CSV-based feedback import and manual issue/feedback management. It will validate records, preserve original text and metadata, store accepted data in PostgreSQL and run a staged analysis pipeline.

Project identity

Item	Description
Project title	Product Feedback Intelligence Platform
Base repository	https://github.com/freefair/issue-tracker
Primary language	Kotlin - backend, domain models, services, APIs and tests
Application type	Self-hosted web application with feedback analytics extensions
Primary users	Product managers, customer-experience teams, support teams and analysts
Core output	Structured feedback, grouped issues, trends, transparent priorities and reports

Central idea: convert scattered customer comments into traceable, organized and prioritized product insights while keeping Kotlin central to implementation.

1. Project: Freefair Issue Tracker

Freefair describes as a self-hosted issue tracker with a Kanban board, backlog management, advanced search and drag-and-drop functionality. Its backend uses Kotlin with Spring Boot and WebFlux. Its frontend uses Next.js, React and TypeScript. PostgreSQL is the required database.

Existing capabilities to retain

Existing feature	Use in this project
Multi-board organization	Create boards for products, product versions or feedback workstreams.
Kanban workflow	Move feedback issues through To Do, In Progress, Ready for Deployment and Done.
Backlog categories	Organize work into Critical, High Priority, Feature Request and Later.
Tags	Label issues by product area, platform, release, sentiment or feedback source.
Advanced search	Find issues by board, tag and status; add feedback-specific filters.
Archive and restore	Keep completed issues available for review and restore recurring issues.
Drag and drop	Reorder tasks and move issues across workflow states.
REST API	Connect dashboard and feedback-analysis modules to Kotlin backend.

Mapping the existing task model

Freefair concept	Project interpretation
Board	Product, product line or feedback workspace
Task	Consolidated feedback issue or feature request
Task description	Issue summary and supporting customer feedback
Tag	Category, sentiment, platform, version or source label
Backlog category	Priority grouping / triage bucket
Status	Investigation and resolution stage

2. Problem Statement and Objectives

Product-based companies receive feedback from app reviews, customer-support tickets, surveys, social comments and feature-request forms. Different customers may describe the same problem in different words. Feedback may contain incomplete metadata or refer to different product versions. As volume grows, spreadsheets and manual reading become difficult to scale.

Problem statement: How can a Kotlin-based application organize product feedback and transform unstructured customer comments into traceable, prioritized and evidence-based product insights?

The project will support decision-making; it will not claim to determine the true root cause automatically. It will surface recurring themes, sentiment and intent signals, unusual changes in feedback volume and evidence behind each candidate issue.

Measurable objectives

No.	Objective	Acceptance evidence
1	Use Freefair board/task workflow to organize product feedback issues.	Boards, statuses, tags and backlog categories work in adapted workflow.
2	Import a documented CSV schema and validate required fields and ratings.	Valid, invalid and malformed fixtures produce expected outcomes.
3	Persist feedback and analysis metadata in PostgreSQL.	Integration tests confirm persistence after restart.
4	Implement baseline sentiment and intent analysis where feasible.	Evaluate precision, recall and F1 on labelled test data where available.
5	Group related feedback and calculate issue counts over time/product segments.	Aggregates match independently checked test-data counts.
6	Flag candidate emerging issues and calculate transparent priority scores.	Tests verify score inputs, ranking and alert thresholds.
7	Provide dashboard filters and exportable intelligence summary.	End-to-end demonstration completes from import to report.

Primary stakeholders

Stakeholder	Need	Platform support
Product manager	Decide what needs attention	Ranked issues, trends, counts and examples
Support / CX	Recognize recurring complaints	Intent labels, issue groups and source filters
Data analyst	Explore consistent data	Validated records, metrics and exports
Engineering team	Notice recurring defects	Trend comparisons and candidate alerts

3. Key Features and Project Scope

The implementation will preserve useful features of the selected tracker and add a product-feedback layer.

A. Existing Freefair foundation

Feature	Description
Boards	Create separate workspaces for products, teams or feedback streams.
Kanban board	Track issues through To Do, In Progress, Ready for Deployment and Done.
Backlog	Store and prioritize work before it enters the active board.
Archive	Review completed work and restore issues when needed.
Tags and search	Label and locate tasks using structured query syntax and tag filters.
Responsive interface	Use the board on desktop and mobile layouts.
REST operations	Create, retrieve, update, move, search and delete boards/tasks through API endpoints.

B. Proposed product-feedback extensions

Planned feature	Purpose
Feedback submission	Allow manual feedback entry or CSV import.
CSV validation	Check required columns, dates, ratings, duplicate IDs and malformed rows.
Feedback-to-issue grouping	Link similar comments to a consolidated issue while preserving source records.
Sentiment and intent	Classify feedback into baseline sentiment and intent labels.
Topic grouping	Identify recurring themes where data and model feasibility permit.
Trend monitoring	Compare issue counts across time windows and flag unusual increases.
Priority scoring	Rank candidate issues using visible, documented factors.
Analytics and export	Show KPIs, issue examples, filters and concise intelligence reports.

Scope boundaries

In scope: CSV ingestion, validation, cleaning, PostgreSQL persistence, Kotlin services and APIs, baseline NLP/ML, issue grouping, time-based trends, explainable priority scoring, dashboard filters, report export and automated testing.

Out of scope for the first release: training a large language model from scratch, guaranteed root-cause or churn prediction, automatic changes to customer accounts/products, every feedback-source integration, production-scale multi-tenant SaaS and unverified claims of model accuracy or business uplift.

4. Technologies Used

The base repository uses a Kotlin-first backend and a separate web frontend. The project will retain that division and implement new business logic primarily in Kotlin. NLP/ML libraries will be selected after compatibility and evaluation experiments.

Layer	Technology	Role in project
Primary language	Kotlin 2.1	Domain models, validation, business logic, processing services, REST API and tests.
Backend framework	Spring Boot 3.4	Application configuration, REST endpoints and service coordination.
Reactive web	Spring WebFlux	Non-blocking request handling.
Async processing	Kotlin Coroutines / Flow	Suspend functions and asynchronous data flows where useful.
Database	PostgreSQL	Boards, tasks, feedback, predictions, issue groups and metrics.
Database access	Spring Data R2DBC	Reactive database access from Kotlin backend.
Migrations	Flyway	Version-controlled database schema changes.
Build	Gradle Kotlin DSL	Build and dependency management.
Runtime	Java 21	JVM runtime for backend application.
Frontend	Next.js 16, React 19, TypeScript 5.9	Dashboard and UI retained from base repository.
UI / interaction	Tailwind CSS, @dnd-kit/react	Responsive styling and drag-and-drop interaction.
Testing	JUnit 5, embedded PostgreSQL, Vitest, Playwright	Backend, frontend and end-to-end tests.
NLP/ML candidates	Apache OpenNLP, Smile; assess ONNX Runtime	Text processing and model inference, subject to compatibility.
Version control	Git and GitHub	Source history and reproducible changes.

Why Kotlin?

Kotlin provides null safety and explicit types for optional feedback fields, ratings and model outputs. Data classes can represent feedback and analysis records; sealed classes can represent accepted and rejected import rows. The JVM ecosystem supports Spring Boot, PostgreSQL drivers and compatible NLP/ML libraries. Coroutines and Flow can support asynchronous operations where justified.

Language-use principle

Kotlin will remain the primary language for backend and core project logic. TypeScript is retained for the existing frontend. A separate Python inference service will not be introduced by default; if a model cannot run suitably on the JVM, the limitation and data flow will be documented before choosing an alternative.

5. System Architecture and Implementation Design

The target design extends the Freefair application rather than replacing its issue-tracking foundation. The Kotlin backend remains responsible for domain logic, validation, persistence, APIs and coordination. The existing Next.js frontend is adapted to present feedback and product insights.

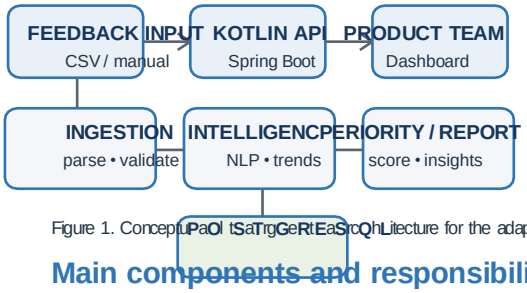


Figure 1. Conceptual architecture for the adapted product feedback platform.

Main components and responsibilities

Component	Input	Responsibility / output
Feedback intake	CSV / manual submission	Parse input, create import record and report row errors.
Validation / cleaning	Parsed feedback rows	Check fields, normalize text/dates, validate ratings and detect duplicates.
Feedback repository	Accepted records	Persist original text, normalized metadata, source and timestamps.
Analysis service	Clean text and metadata	Produce sentiment/intent labels and optional topic signals.
Issue grouping	Feedback and analysis	Associate related feedback with consolidated issues and preserve traceability.
Trend service	Dated feedback / counts	Compare time windows and flag unusual changes.
Priority engine	Volume, growth and impact proxies	Calculate transparent score and store contributing factors.
Kotlin REST API	Stored records and aggregates	Serve workflows, filters, issue details, alerts and report data.
Dashboard	API responses	Display boards, KPIs, trends, priorities and exportable summaries.

Implementation principle

Implement reliable ingestion, validation, persistence and issue workflow first. Add baseline analytics next. Increase model complexity only when the dataset, JVM compatibility and evaluation results justify it.

6. Data Flow and Data Model

The workflow starts with feedback import or manual entry. Accepted records are stored with original text and metadata. Analysis results are stored separately so source feedback is not overwritten by predicted labels. Related records can then be associated with a tracker task or consolidated issue.

Step	Action	Result
1. Collect	Upload CSV or enter feedback manually.	Import job / feedback submission created.
2. Validate	Check schema, required fields, ratings, dates and duplicates.	Accepted records and rejected-row reasons.
3. Store	Save original feedback and normalized metadata.	Persistent feedback records in PostgreSQL.
4. Analyze	Run baseline sentiment/intent and optional topic grouping.	Analysis results with model/version metadata.
5. Organize	Link related records to issue/task and add tags/categories.	Traceable issue groups in tracker.
6. Monitor	Aggregate counts by product, segment and time window.	Trend metrics and candidate alerts.
7. Prioritize	Apply documented score components.	Ranked issue list with explanation.
8. Report	Display filters, evidence and summary metrics.	Dashboard and exportable report.

Proposed data entities

Entity	Important fields	Purpose
FeedbackRecord	id, text, timestamp, product, rating?, source, segment?	Original feedback and normalized metadata.
AnalysisResult	feedbackId, sentiment?, intent?, topicId?, confidence?, modelVersion	Derived labels kept separate from source data.
IssueCluster	topicId, label, createdAt, currentVolume, trendStatus	Group of related feedback records.
IssueMetric	topicId, timeWindow, count, baselineCount, growthRate?	Time-based comparisons and alerts.
PriorityAssessment	topicId, score, factors, assessedAt, ruleVersion	Reproducible issue ranking.
ImportJob	id, filename, status, rowsRead, rowsAccepted, rowsRejected	Tracks each CSV import.
Tracker Task	id, boardId, title, description, status, tags, backlogCategoryId	Existing Freefair workflow for consolidated issues.

Kotlin model sketch (illustrative)

```
data class FeedbackRecord(val id: String, val text: String, val timestamp: Instant?, val product: String?, val rating: Int?, val source: String)
sealed interface RowResult { data class Accepted(val record: FeedbackRecord) : RowResult; data class Rejected(val rowNumber: Int, val reason: String) : RowResult }
```

Missing values remain explicitly optional. Import metadata, model versions and scoring-rule versions will be retained for reproducibility.

7. Feedback Intelligence and Prioritization

The intelligence layer turns feedback records into signals that help a product team decide what to investigate. The first implementation will use understandable baselines and preserve evidence for each result. Predictions will not be presented as certain facts.

Analysis capabilities

Capability	Proposed method	Output
Sentiment analysis	JVM-compatible baseline classifier or evaluated model.	Positive, neutral or negative signal.
Intent classification	Rules or supervised baseline depending on labelled data.	Bug report, feature request, usability issue, praise or supported category.
Topic / issue grouping	Text similarity or clustering after data-quality review.	Candidate groups of related feedback.
Trend monitoring	Compare counts across current and prior windows.	Trend charts and candidate emerging-issue alerts.
Duplicate detection	Compare normalized text and similarity signals.	Possible duplicate records for human review.
Priority scoring	Deterministic weighted score with visible factors.	Ranked candidate issue list.

Explainable priority score

Candidate formula: Priority Score = w_v × Volume + w_g × Growth + w_s × Severity Signal + w_i × Impact Proxy

Volume measures how many records relate to an issue. Growth compares the current time window with a prior baseline. Severity may use defined issue labels or sentiment intensity. Impact proxy may use available product or segment metadata. Weights and thresholds will be documented and tested; this formula is a triage aid, not a validated business-risk measure.

Illustrative examples

Issue group	Volume	Growth	Interpretation
Checkout failure	High	Increasing	Investigate early; inspect examples and affected versions.
Dark-mode request	Medium	Stable	Keep in feature backlog and review with roadmap.
Minor wording issue	Low	Stable	Lower urgency unless new evidence appears.

Evaluation of model outputs

When labelled data is available, sentiment and intent predictions will be assessed using precision, recall, F1 and a confusion matrix. Topic groups will be inspected for coherence and representative examples. Trend alerts will be tested with stable and sudden-spike synthetic time series. Priority scores will be tested with hand-calculated cases and boundary conditions.

8. Privacy, Risks, Testing and Work Plan

Privacy and responsible use

The prototype will use public, appropriately licensed, anonymized or synthetic feedback wherever possible. It will not assume permission to process private company data. Feedback will not be sent to an external model provider unless explicitly configured and disclosed. Sentiment and topic labels are imperfect model outputs, not facts about a customer's personal traits or intent.

Risk	Potential effect	Mitigation
Personal data in feedback	Privacy exposure	Prefer anonymized/public data; minimize stored fields; document retention and access.
Noisy or biased labels	Misleading sentiment/intent	Evaluate by class, inspect errors and document dataset limits.
False trend alert	Unnecessary investigation	Use baselines, minimum-volume thresholds and visible evidence.
Misleading priority score	Over-trust in ranking	Show score factors and rule version; call it triage support.
Malformed CSV	Import failure or silent loss	Validate headers and report rejected rows with reasons.
Base repository security	Unauthorized access if exposed	Review defaults; add authentication, HTTPS and rate limits before deployment.

Testing plan

Test area	Method / evidence
CSV and validation	Test valid, empty, malformed, missing-column and unexpected-value files.
Analysis	Compare predictions against labelled examples; report precision, recall and F1 where possible.
Trends and score	Use synthetic stable/spike series and hand-calculated priority examples.
Database and API	Integration tests, endpoint tests and restart-persistence checks.
Tracker workflow	Test board/task creation, status moves, tags, backlog, search and archive.
End-to-end	Import feedback, inspect grouped issues, filter, prioritize and export report.

Indicative eight-week implementation plan

Week	Milestone
1	Clone and run Freefair; inspect Kotlin domain, services, API and database.
2	Adapt boards/tasks/tags and define feedback CSV schema.
3	Implement CSV ingestion, validation and persistence.
4	Add baseline sentiment and intent analysis.
5	Add grouping, trends, emerging-issue rules and priority scoring.
6	Adapt dashboard and add analytics/report views.
7	Testing, security review, error handling and setup documentation.
8	Evaluation, final demo, report and viva preparation.

9. Implementation Approach, Expected Outcomes and Conclusion

Implementation approach

The work will begin by cloning the selected repository and establishing a reproducible local run. Existing board, task, backlog, tag and search features will be inspected before modification. The Kotlin backend will then be extended with feedback-specific domain models, import services, analysis services, repositories and API endpoints. Database changes will be introduced through new Flyway migrations rather than editing previously applied migrations.

The frontend will be adapted to expose feedback entry, issue evidence and analytics while retaining the existing workflow patterns. The repository's documented local setup uses Java 21, Node.js 24+ and PostgreSQL 12+; its README also describes Docker-based startup. Follow the repository README for exact commands and environment variables because setup details can change.

Expected outcomes

Deliverable	Completion evidence
Adapted Kotlin application	Freefair tracker runs with product/feedback-oriented boards and issue labels.
Data pipeline	Sample CSV imports; invalid rows are explained.
Feedback intelligence	Documented sentiment/intent labels, issue groups and trend metrics.
Prioritization	Ranked issues show score factors and rule version.
Dashboard and report	Filters, issue summaries, supporting counts and export workflow.
Quality evidence	Automated tests, evaluation results where supported, limitations and setup guide.

Conclusion

The Product Feedback Intelligence Platform will adapt a Kotlin/Spring Boot issue-tracking application into a product-feedback workflow. Freefair supplies a reusable foundation for boards, tasks, backlog categories, tags, search, archive, REST APIs and PostgreSQL persistence. The project-specific contribution is the addition of feedback ingestion, data validation, NLP/ML-derived signals, issue grouping, trend monitoring, transparent prioritization and reporting.

Kotlin remains central to the backend, data models, processing services, API integration and tests. The project will distinguish existing repository functionality from new development, evaluate analytics rather than assume accuracy, and preserve source feedback so product teams can trace insights to evidence.

References

[1] JetBrains, “Kotlin Documentation.” https://kotlinlang.org/docs/home.html.

[2] Spring, “Spring Boot Reference Documentation.” https://docs.spring.io/spring-boot/index.html.

[3] PostgreSQL Global Development Group, “PostgreSQL Documentation.” https://www.postgresql.org/docs/current/.

[4] Apache Software Foundation, “Apache OpenNLP.” https://opennlp.apache.org/.

[5] Tribuo Contributors, “Tribuo: Machine Learning in Java.” https://tribuo.org/.

[6] Google, “Rules of Machine Learning.” https://developers.google.com/machine-learning/guides/rules-of-ml.

[7] Scikit-learn developers, “Model Evaluation.” https://scikit-learn.org/stable/modules/model_evaluation.html.